Modelos de Linguagem de Grande Escala (LLMs)

Aula 8: como um LLM funciona por dentro

Prof. Elton Sarmanho[1]
eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA Campus Cametá

18 de outubro de 2026

github.com/eltonsarmanho Elton Sarmanho

Roteiro da Aula

- Planejamento
- O que é um LLM?
  - Modelo de linguagem
  - Uma breve história
- Tokens e Embeddings
  - Tokenização
  - Embeddings
- Atenção e Transformer
  - Autoatenção
  - O bloco Transformer

Roteiro da Aula

Como um LLM é treinado

Como o texto é gerado
- Decodificação

Prompting e Limites
- Técnicas de prompt
- Limites

Python para LLMs

Referências Bibliográficas

Nesta aula

- Entender o que um LLM realmente calcula: a **probabilidade do próximo token**.
- Ver como o texto vira **tokens** e vetores, e como a **atenção** mistura o contexto.
- Entender as decisões de geração: **temperatura**, *top-k* e *top-p*.
- Usar *prompts* com critério e reconhecer os **limites** que motivam o RAG (Aula 7).
- Todos os números desta aula vêm de execuções reais do `llama3.2` no notebook `LLM.ipynb`.

Reflexão

Na Aula 7, o RAG entregava evidências a um LLM. Mas o que o LLM faz com elas, token a token?

O que é um LLM?

Modelo de Linguagem

Definição

Um **modelo de linguagem** atribui probabilidades a sequências de texto. Na forma usada pelos LLMs, ele estima a probabilidade de cada **próximo token** dado tudo o que veio antes (*autorregressivo*).

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P_\theta(w_t \mid w_1, \ldots, w_{t-1})$$

w_t: token na posição t; $w_1, \ldots, w_{t-1}$: contexto; θ: parâmetros (pesos) do modelo.

Um LLM (*Large Language Model*) é um modelo de linguagem com bilhões de parâmetros, treinado em grandes coleções de texto. O `llama3.2` usado na aula tem cerca de 3 bilhões (3B).

Modelo de Linguagem

Definição

Um **modelo de linguagem** atribui probabilidades a sequências de texto. Na forma usada pelos LLMs, ele estima a probabilidade de cada **próximo token** dado tudo o que veio antes (*autorregressivo*).

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P_\theta(w_t \mid w_1, \ldots, w_{t-1})$$

w_t: token na posição t; $w_1, \ldots, w_{t-1}$: contexto; θ: parâmetros (pesos) do modelo.

Um LLM (*Large Language Model*) é um modelo de linguagem com bilhões de parâmetros, treinado em grandes coleções de texto. O `llama3.2` usado na aula tem cerca de 3 bilhões (3B).

Modelo de Linguagem

Definição

Um **modelo de linguagem** atribui probabilidades a sequências de texto. Na forma usada pelos LLMs, ele estima a probabilidade de cada **próximo token** dado tudo o que veio antes (*autorregressivo*).

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P_\theta(w_t \mid w_1, \ldots, w_{t-1})$$

w_t: token na posição t; $w_1, \ldots, w_{t-1}$: contexto; θ: parâmetros (pesos) do modelo.

Um LLM (*Large Language Model*) é um modelo de linguagem com bilhões de parâmetros, treinado em grandes coleções de texto. O `llama3.2` usado na aula tem cerca de 3 bilhões (3B).

Analogia

Autocompletar do celular

Olha as últimas uma ou duas palavras e sugere a próxima, a partir de contagens.

LLM

Olha **todo** o contexto (milhares de tokens) e calcula uma distribuição de probabilidade sobre as 128.256 peças do vocabulário.

O ponto central

O LLM não consulta um banco de fatos: ele escolhe tokens prováveis. Quando o fato está bem representado no treino, o provável coincide com o verdadeiro; quando não está, o provável pode ser falso.

Analogia

Autocompletar do celular

Olha as últimas uma ou duas palavras e sugere a próxima, a partir de contagens.

LLM

Olha **todo** o contexto (milhares de tokens) e calcula uma distribuição de probabilidade sobre as 128.256 peças do vocabulário.

O ponto central

O LLM não consulta um banco de fatos: ele escolhe tokens prováveis. Quando o fato está bem representado no treino, o provável coincide com o verdadeiro; quando não está, o provável pode ser falso.

Analogia

Autocompletar do celular

Olha as últimas uma ou duas palavras e sugere a próxima, a partir de contagens.

LLM

Olha **todo** o contexto (milhares de tokens) e calcula uma distribuição de probabilidade sobre as 128.256 peças do vocabulário.

O ponto central

O LLM não consulta um banco de fatos: ele escolhe tokens prováveis. Quando o fato está bem representado no treino, o provável coincide com o verdadeiro; quando não está, o provável pode ser falso.

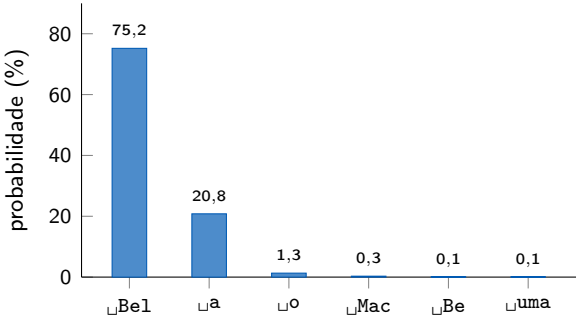
Na prática: o próximo token

Contexto: *"A capital do Pará é"*

Tokens: `A` `␣capital` `␣do` `␣Par` `á` `␣é`

O modelo devolve uma probabilidade para cada token do vocabulário. O mais provável é `␣Bel`, com **75,2%**; depois vem `␣a`, com 20,8%.

execução real · llama3.2 (Ollama, CPU) · *seed* 42



De contagens a Transformers



O que mudou em 2017

A **atenção** permite que cada token consulte diretamente todos os anteriores, em paralelo. Isso viabilizou treinar modelos muito maiores, em muito mais texto.

Tokens e Embeddings

Tokenização: o texto vira peças

BPE — *Byte Pair Encoding* (Sennrich et al., 2016)

Começa com bytes e junta, repetidamente, o par mais frequente no corpus de treino. Palavras comuns viram um único token; palavras raras são quebradas em subpalavras. O vocabulário do `llama3.2` tem **128.256** tokens.

Texto	Tokens (`llama3.2`)	nº
Inteligência Artificial	`Int` `elig` `ência` `␣Artificial`	4
heurística	`he` `ur` `ística`	3
Cametá	`Cam` `et` `á`	3
Sistemas de Informação	`S` `ist` `emas` `␣de` `␣Inform` `ação`	6

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Tokenização: o texto vira peças

BPE — *Byte Pair Encoding* (Sennrich et al., 2016)

Começa com bytes e junta, repetidamente, o par mais frequente no corpus de treino. Palavras comuns viram um único token; palavras raras são quebradas em subpalavras. O vocabulário do `llama3.2` tem **128.256** tokens.

Texto	**Tokens** (`llama3.2`)	**nº**
Inteligência Artificial	`Int` `elig` `ência` `␣Artificial`	4
heurística	`he` `ur` `ística`	3
Cametá	`Cam` `et` `á`	3
Sistemas de Informação	`S` `ist` `emas` `␣de` `␣Inform` `ação`	6

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Por que tokens importam

15 tokens (10 palavras): `A` `␣carga` `␣hor` `ária` `␣total` `␣do` `␣curso` `␣é` `␣de` `␣` `3` `.` `332` `␣horas` `.`

13 tokens (9 palavras): `The` `␣total` `␣workload` `␣of` `␣the` `␣course` `␣is` `␣` `3` `,` `332` `␣hours` `.`

Consequências

- Custo e limite são medidos em **tokens**, não em palavras.
- Aqui, a frase em português gastou mais tokens que a mesma frase em inglês (15×13). É comum: o vocabulário foi construído com mais texto em inglês.
- Números são cortados em grupos de até 3 dígitos: `3` `.` `332`.

Janela de contexto

O `llama3.2` foi treinado para até **131.072** tokens de contexto. O PPC inteiro tem 79.827 tokens. Mas a janela que o Ollama usou por padrão neste PC é bem menor: de 14.174 tokens enviados, processou só os últimos 2.050; com `num_ctx = 16384`, processou os 14.174. O que não cabe na janela, o modelo não vê.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Por que tokens importam

15 tokens (10 palavras): `A` `␣carga` `␣hor` `ária` `␣total` `␣do` `␣curso` `␣é` `␣de` `␣` `3` `.` `332` `␣horas` `.`

13 tokens (9 palavras): `The` `␣total` `␣workload` `␣of` `␣the` `␣course` `␣is` `␣` `3` `,` `332` `␣hours` `.`

Consequências

- Custo e limite são medidos em **tokens**, não em palavras.
- Aqui, a frase em português gastou mais tokens que a mesma frase em inglês (15×13). É comum: o vocabulário foi construído com mais texto em inglês.
- Números são cortados em grupos de até 3 dígitos: `3` `.` `332`.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Janela de contexto

O `llama3.2` foi treinado para até **131.072** tokens de contexto. O PPC inteiro tem 79.827 tokens. Mas a janela que o Ollama usou por padrão neste PC é bem menor: de 14.174 tokens enviados, processou só os últimos 2.050; com `num_ctx = 16384`, processou os 14.174. O que não cabe na janela, o modelo não vê.

De tokens a vetores

Tabela de *embeddings*

Cada token é um número (o seu *id*) que indexa uma linha de uma tabela aprendida: um vetor de **3.072** dimensões no `llama3.2`.

`␣capital`

id → linha da tabela

0,12	-0,4	0,9	0,05	-0,7	···

vetor de 3.072 números

(valores ilustrativos)

Posição

O modelo também precisa saber a **ordem**. O `llama3.2` codifica a posição girando os vetores de consulta e chave (*RoPE*, Su et al., 2021).

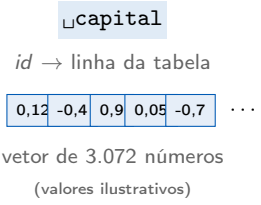
Reflexão

Na Aula 7, os *embeddings* do MiniLM representavam frases inteiras. Aqui, cada token tem o seu vetor. Qual é a diferença de uso?

De tokens a vetores

Tabela de *embeddings*

Cada token é um número (o seu *id*) que indexa uma linha de uma tabela aprendida: um vetor de **3.072** dimensões no `llama3.2`.



Posição

O modelo também precisa saber a **ordem**. O `llama3.2` codifica a posição girando os vetores de consulta e chave (*RoPE*, Su et al., 2021).

Reflexão

Na Aula 7, os *embeddings* do MiniLM representavam frases inteiras. Aqui, cada token tem o seu vetor. Qual é a diferença de uso?

Atenção e Transformer

Autoatenção (*self-attention*)

Ideia (Vaswani et al., 2017)

Cada token gera três vetores: **consulta** q (o que procuro), **chave** k (o que ofereço) e **valor** v (o que entrego). A consulta é comparada com as chaves de todos os tokens anteriores; os pesos resultantes misturam os valores.

$$\text{Atenção}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

$QK^\top$: similaridade consulta $\times$ chave; $\sqrt{d_k}$: escala pela dimensão; M: **máscara causal** ($-\infty$ para tokens futuros); softmax: transforma em pesos que somam 1.

A máscara causal é o que faz o modelo ser autorregressivo: ao prever a posição t, ele só enxerga $w_1, \ldots, w_{t-1}$.

Autoatenção (*self-attention*)

Ideia (Vaswani et al., 2017)

Cada token gera três vetores: **consulta** q (o que procuro), **chave** k (o que ofereço) e **valor** v (o que entrego). A consulta é comparada com as chaves de todos os tokens anteriores; os pesos resultantes misturam os valores.

$$\text{Atenção}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

$QK^\top$: similaridade consulta $\times$ chave; $\sqrt{d_k}$: escala pela dimensão; M: **máscara causal** ($-\infty$ para tokens futuros); softmax: transforma em pesos que somam 1.

A máscara causal é o que faz o modelo ser autorregressivo: ao prever a posição t, ele só enxerga $w_1, \ldots, w_{t-1}$.

Autoatenção (*self-attention*)

Ideia (Vaswani et al., 2017)

Cada token gera três vetores: **consulta** q (o que procuro), **chave** k (o que ofereço) e **valor** v (o que entrego). A consulta é comparada com as chaves de todos os tokens anteriores; os pesos resultantes misturam os valores.

$$\text{Atenção}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

$QK^\top$: similaridade consulta $\times$ chave; $\sqrt{d_k}$: escala pela dimensão; M: **máscara causal** ($-\infty$ para tokens futuros); softmax: transforma em pesos que somam 1.

A máscara causal é o que faz o modelo ser autorregressivo: ao prever a posição t, ele só enxerga $w_1, \ldots, w_{t-1}$.

Questão

Considere o contexto *"capital do Pará"* e a consulta do último token, `Pará`, com $d_k = 4$:

$$q_{\text{Pará}} = (1, 0, 1, 0)$$

token	k	v
capital	$(1, 0, 1, 0)$	$(1, 0)$
do	$(0, 1, 0, 0)$	$(0, 1)$
Pará	$(1, 1, 0, 0)$	$(1, 1)$

Questão:

1. Calcule os escores $q \cdot k_j / \sqrt{d_k}$ para os três tokens.
2. Aplique a softmax para obter os pesos de atenção.
3. Calcule a saída $\sum_j \alpha_j v_j$. Qual token mais influencia `Pará`?

Dicas para Resolução

Passos para resolver:

- Produto escalar: some os produtos componente a componente.
- Divida por $\sqrt{4} = 2$.
- Softmax: $\alpha_j = e^{s_j} / \sum_i e^{s_i}$. Use $e^1 \approx 2{,}718$, $e^{0{,}5} \approx 1{,}649$ e $e^0 = 1$.
- A saída é a média dos valores v_j, ponderada pelos pesos α_j.

Passo 1: Escores

$$
\begin{aligned}
q \cdot k_{\text{capital}} &= 1{\cdot}1 + 0{\cdot}0 + 1{\cdot}1 + 0{\cdot}0 = 2 && \Rightarrow\ s_1 = 2/2 = 1 \\
q \cdot k_{\text{do}} &= 1{\cdot}0 + 0{\cdot}1 + 1{\cdot}0 + 0{\cdot}0 = 0 && \Rightarrow\ s_2 = 0/2 = 0 \\
q \cdot k_{\text{Pará}} &= 1{\cdot}1 + 0{\cdot}1 + 1{\cdot}0 + 0{\cdot}0 = 1 && \Rightarrow\ s_3 = 1/2 = 0{,}5
\end{aligned}
$$

Leitura

A chave de *capital* é a mais parecida com a consulta. A máscara causal não atua aqui: *Pará* é o último token e enxerga todos os anteriores (e a si mesmo).

Passo 1: Escores

$$
\begin{aligned}
q \cdot k_{\text{capital}} &= 1{\cdot}1 + 0{\cdot}0 + 1{\cdot}1 + 0{\cdot}0 = 2 && \Rightarrow\ s_1 = 2/2 = 1 \\
q \cdot k_{\text{do}} &= 1{\cdot}0 + 0{\cdot}1 + 1{\cdot}0 + 0{\cdot}0 = 0 && \Rightarrow\ s_2 = 0/2 = 0 \\
q \cdot k_{\text{Pará}} &= 1{\cdot}1 + 0{\cdot}1 + 1{\cdot}0 + 0{\cdot}0 = 1 && \Rightarrow\ s_3 = 1/2 = 0{,}5
\end{aligned}
$$

Leitura

A chave de *capital* é a mais parecida com a consulta. A máscara causal não atua aqui: *Pará* é o último token e enxerga todos os anteriores (e a si mesmo).

Passo 2: Softmax

$$\sum_i e^{s_i} = e^1 + e^0 + e^{0,5} \approx 2,718 + 1 + 1,649 = 5,367$$

$$\alpha_{\text{capital}} \approx \frac{2,718}{5,367} \approx 0,506 \qquad \alpha_{\text{do}} \approx \frac{1}{5,367} \approx 0,186 \qquad \alpha_{\text{Pará}} \approx \frac{1,649}{5,367} \approx 0,307$$

pesos de *Pará*:

0,506	0,186	0,307
capital	do	Pará

Os pesos somam 1 (com arredondamento).

Passo 2: Softmax

$$\sum_i e^{s_i} = e^1 + e^0 + e^{0,5} \approx 2,718 + 1 + 1,649 = 5,367$$

$$\alpha_{\text{capital}} \approx \frac{2,718}{5,367} \approx 0,506 \qquad \alpha_{\text{do}} \approx \frac{1}{5,367} \approx 0,186 \qquad \alpha_{\text{Pará}} \approx \frac{1,649}{5,367} \approx 0,307$$

pesos de *Pará*:

0,506	0,186	0,307
capital	do	Pará

Os pesos somam 1 (com arredondamento).

Passo 3: Saída ponderada

$$o_{\text{Pará}} = 0{,}506\,(1,0) + 0{,}186\,(0,1) + 0{,}307\,(1,1) \approx (0{,}814;\ 0{,}494)$$

Interpretação

A nova representação de *Pará* é, em grande parte, informação vinda de *capital*: o contexto foi **misturado** ao token. Em um modelo real, isso se repete em muitas cabeças e camadas.

No `llama3.2`

28 camadas, 24 cabeças de atenção por camada, compartilhando 8 grupos de chave e valor (*grouped-query attention*). Os vetores têm 3.072 dimensões. Valores lidos do próprio arquivo do modelo.

Passo 3: Saída ponderada

$$o_{\text{Pará}} = 0{,}506\,(1,0) + 0{,}186\,(0,1) + 0{,}307\,(1,1) \approx (0{,}814;\ 0{,}494)$$

Interpretação

A nova representação de *Pará* é, em grande parte, informação vinda de *capital*: o contexto foi **misturado** ao token. Em um modelo real, isso se repete em muitas cabeças e camadas.

No `llama3.2`

28 camadas, 24 cabeças de atenção por camada, compartilhando 8 grupos de chave e valor (*grouped-query attention*). Os vetores têm 3.072 dimensões. Valores lidos do próprio arquivo do modelo.

Passo 3: Saída ponderada

$$o_{\text{Pará}} = 0{,}506\,(1,0) + 0{,}186\,(0,1) + 0{,}307\,(1,1) \approx (0{,}814;\ 0{,}494)$$

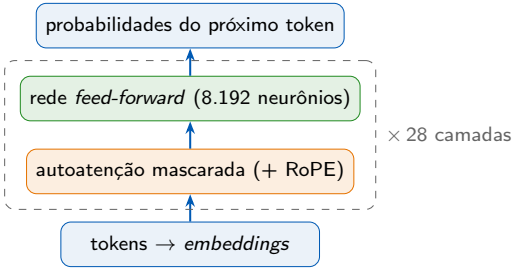
Interpretação

A nova representação de *Pará* é, em grande parte, informação vinda de *capital*: o contexto foi **misturado** ao token. Em um modelo real, isso se repete em muitas cabeças e camadas.

No `llama3.2`

28 camadas, 24 cabeças de atenção por camada, compartilhando 8 grupos de chave e valor (*grouped-query attention*). Os vetores têm 3.072 dimensões. Valores lidos do próprio arquivo do modelo.

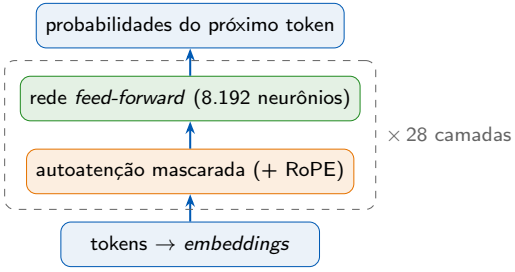
O Transformer decodificador



Cada camada

A atenção mistura informação **entre** tokens; a *feed-forward* transforma cada token **individualmente**. Conexões residuais e normalização mantêm o treino estável. A última camada projeta o vetor em 128.256 escores, um por token do vocabulário.

O Transformer decodificador

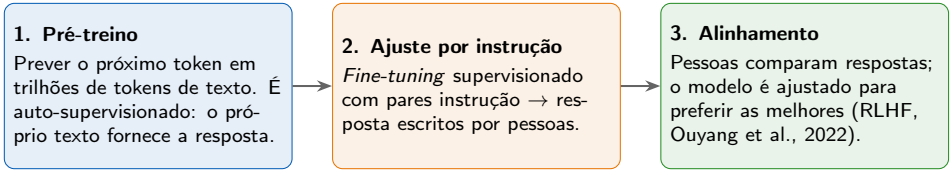


Cada camada

A atenção mistura informação **entre** tokens; a *feed-forward* transforma cada token **individualmente**. Conexões residuais e normalização mantêm o treino estável. A última camada projeta o vetor em 128.256 escores, um por token do vocabulário.

Como um LLM é treinado

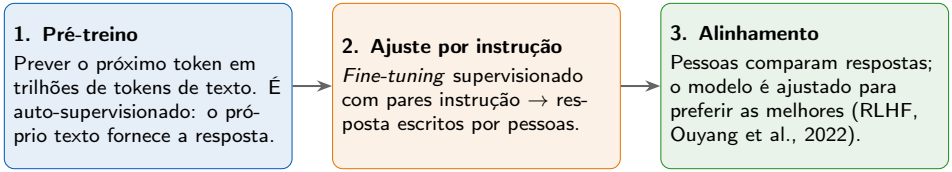
Três etapas



$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{t=1}^{N} \log P_\theta(w_t \mid w_{<t})$$

Entropia cruzada: penaliza o modelo quando ele dá pouca probabilidade ao token que de fato veio. Minimizá-la é o objetivo do pré-treino.

Três etapas



$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{t=1}^{N} \log P_\theta(w_t \mid w_{<t})$$

Entropia cruzada: penaliza o modelo quando ele dá pouca probabilidade ao token que de fato veio. Minimizá-la é o objetivo do pré-treino.

Modelo base × modelo instruído

Continuação crua (modo `raw`)

"A carga horária total do curso de Sistemas de Informação da UFPA Campus Cametá é de" → **"600 horas"**.

Conversa (modelo instruído)

"Qual a carga horária total do curso. . . ?" → **"Desculpe, mas não consegui encontrar informações atualizadas. . ."**

É o mesmo modelo. Como completador de texto, ele inventa um número; no formato de conversa, o ajuste por instrução o leva a admitir que não sabe. O valor correto, no PPC, é **3.332 horas**.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Reflexão

Por que o mesmo modelo responde de formas tão diferentes só mudando o formato da entrada?

Modelo base × modelo instruído

Continuação crua (modo `raw`)

"A carga horária total do curso de Sistemas de Informação da UFPA Campus Cametá é de" → **"600 horas"**.

Conversa (modelo instruído)

"Qual a carga horária total do curso. . . ?" → **"Desculpe, mas não consegui encontrar informações atualizadas. . ."**

É o mesmo modelo. Como completador de texto, ele inventa um número; no formato de conversa, o ajuste por instrução o leva a admitir que não sabe. O valor correto, no PPC, é **3.332 horas**.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Reflexão

Por que o mesmo modelo responde de formas tão diferentes só mudando o formato da entrada?

Escala

Leis de escala

A perda cai de forma previsível quando aumentam parâmetros, dados e computação (Kaplan et al., 2020). Para um orçamento fixo de computação, o melhor é crescer parâmetros e tokens de treino na mesma proporção, cerca de 20 tokens por parâmetro (Hoffmann et al., 2022).

Modelos pequenos e locais

O `llama3.2` (3B) roda num notebook comum via Ollama. É ótimo para aprender e para dados que não podem sair da máquina, mas erra mais que modelos maiores. Avalie sempre na sua tarefa.

Escala

Leis de escala

A perda cai de forma previsível quando aumentam parâmetros, dados e computação (Kaplan et al., 2020). Para um orçamento fixo de computação, o melhor é crescer parâmetros e tokens de treino na mesma proporção, cerca de 20 tokens por parâmetro (Hoffmann et al., 2022).

Modelos pequenos e locais

O `llama3.2` (3B) roda num notebook comum via Ollama. É ótimo para aprender e para dados que não podem sair da máquina, mas erra mais que modelos maiores. Avalie sempre na sua tarefa.

Como o texto é gerado

Decodificação: escolher o próximo token

Gulosa (*greedy*)

Sempre escolhe o token mais provável. É determinística.

"A capital do Pará é" → *"Belém, a maior cidade do estado"*.

Amostragem

Sorteia o próximo token de acordo com as probabilidades. Gera variedade, mas pode escolher tokens improváveis.

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

z_i: escore (*logit*) do token i; T: **temperatura**. $T < 1$ concentra a probabilidade no favorito; $T > 1$ espalha. $T \to 0$ equivale à gulosa.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Decodificação: escolher o próximo token

Gulosa (*greedy*)

Sempre escolhe o token mais provável. É determinística.

"A capital do Pará é" → *"Belém, a maior cidade do estado"*.

Amostragem

Sorteia o próximo token de acordo com as probabilidades. Gera variedade, mas pode escolher tokens improváveis.

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

z_i: escore (*logit*) do token i; T: **temperatura**. $T < 1$ concentra a probabilidade no favorito; $T > 1$ espalha. $T \to 0$ equivale à gulosa.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Decodificação: escolher o próximo token

Gulosa (*greedy*)

Sempre escolhe o token mais provável. É determinística.

"A capital do Pará é" → *"Belém, a maior cidade do estado"*.

Amostragem

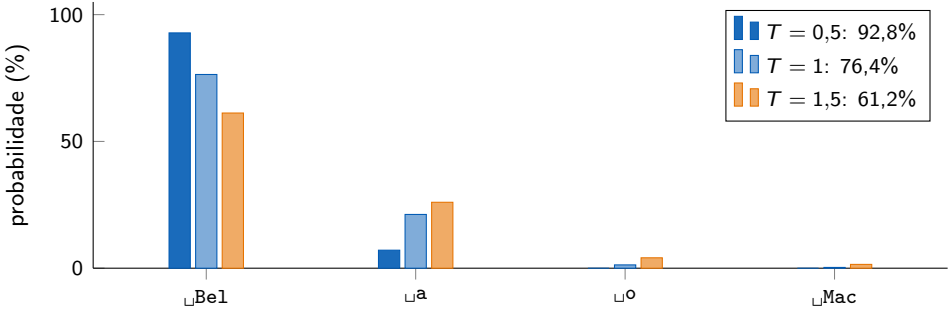
Sorteia o próximo token de acordo com as probabilidades. Gera variedade, mas pode escolher tokens improváveis.

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

z_i: escore (*logit*) do token i; T: **temperatura**. $T < 1$ concentra a probabilidade no favorito; $T > 1$ espalha. $T \to 0$ equivale à gulosa.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Temperatura na distribuição real



Probabilidade de `␣Bel` após *"A capital do Pará é"*, recalculada entre os 20 tokens mais prováveis (que somam 98,4% da massa).

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Top-k e *top-p* (núcleo)

Top-k

Sorteia só entre os k tokens mais prováveis.

Top-p (Holtzman et al., 2020)

Sorteia só no menor conjunto de tokens cuja probabilidade somada atinge p. O tamanho do conjunto se adapta à confiança do modelo.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

No exemplo real ($p = 0{,}9$)

O núcleo tem só **2 tokens**: ␣Bel, ␣a, que somam 96,0%. Todos os outros 128.256 menos 2 ficam de fora.

Reflexão

Num contexto em que o modelo está inseguro, o núcleo de $p = 0{,}9$ fica maior ou menor?

Top-k e *top-p* (núcleo)

Top-k

Sorteia só entre os k tokens mais prováveis.

Top-p (Holtzman et al., 2020)

Sorteia só no menor conjunto de tokens cuja probabilidade somada atinge p. O tamanho do conjunto se adapta à confiança do modelo.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

No exemplo real ($p = 0{,}9$)

O núcleo tem só **2 tokens**: ␣Bel, ␣a, que somam 96,0%. Todos os outros 128.256 menos 2 ficam de fora.

Reflexão

Num contexto em que o modelo está inseguro, o núcleo de $p = 0{,}9$ fica maior ou menor?

Questão

Um modelo produz os escores $z = (2,\ 1,\ 0)$ para os tokens A, B e C. **Questão:**

1. Calcule as probabilidades com $T = 1$.
2. Calcule as probabilidades com $T = 0{,}5$.
3. Com $T = 1$ e *top-p* $= 0{,}9$, quais tokens podem ser sorteados?

Dicas para resolução

- Divida os escores por T antes da exponencial.
- $e^4 \approx 54{,}598$; $e^2 \approx 7{,}389$; $e^1 \approx 2{,}718$; $e^0 = 1$.
- Para o *top-p*, ordene e some as probabilidades até atingir 0,9.

Questão

Um modelo produz os escores $z = (2,\ 1,\ 0)$ para os tokens A, B e C. **Questão:**

1. Calcule as probabilidades com $T = 1$.
2. Calcule as probabilidades com $T = 0{,}5$.
3. Com $T = 1$ e *top-p* $= 0{,}9$, quais tokens podem ser sorteados?

Dicas para resolução

- Divida os escores por T antes da exponencial.
- $e^4 \approx 54{,}598$; $e^2 \approx 7{,}389$; $e^1 \approx 2{,}718$; $e^0 = 1$.
- Para o *top-p*, ordene e some as probabilidades até atingir 0,9.

Resolução

Passo 1 ($T = 1$): soma $= 7{,}389 + 2{,}718 + 1 = 11{,}107$

$$p_A \approx 0{,}665 \qquad p_B \approx 0{,}245 \qquad p_C \approx 0{,}090$$

Passo 2 ($T = 0{,}5$): escores viram $(4, 2, 0)$; soma $= 54{,}598 + 7{,}389 + 1 = 62{,}987$

$$p_A \approx 0{,}867 \qquad p_B \approx 0{,}117 \qquad p_C \approx 0{,}016$$

Passo 3 ($top\text{-}p = 0{,}9$, com $T = 1$): $0{,}665 < 0{,}9$; $0{,}665 + 0{,}245 = 0{,}910 \geq 0{,}9$. O núcleo é $\{A, B\}$.

Temperatura menor aumentou a vantagem de A (de 0,665 para 0,867): o mesmo efeito que vimos com `␣Bel`.

Resolução

Passo 1 ($T = 1$): soma $= 7{,}389 + 2{,}718 + 1 = 11{,}107$

$$p_A \approx 0{,}665 \qquad p_B \approx 0{,}245 \qquad p_C \approx 0{,}090$$

Passo 2 ($T = 0{,}5$): escores viram $(4, 2, 0)$; soma $= 54{,}598 + 7{,}389 + 1 = 62{,}987$

$$p_A \approx 0{,}867 \qquad p_B \approx 0{,}117 \qquad p_C \approx 0{,}016$$

Passo 3 ($\textit{top-}p = 0{,}9$, com $T = 1$): $0{,}665 < 0{,}9$; $0{,}665 + 0{,}245 = 0{,}910 \geq 0{,}9$. O núcleo é $\{A, B\}$.

Temperatura menor aumentou a vantagem de A (de 0,665 para 0,867): o mesmo efeito que vimos com `␣Bel`.

Resolução

Passo 1 ($T = 1$): soma $= 7{,}389 + 2{,}718 + 1 = 11{,}107$

$$p_A \approx 0{,}665 \qquad p_B \approx 0{,}245 \qquad p_C \approx 0{,}090$$

Passo 2 ($T = 0{,}5$): escores viram $(4, 2, 0)$; soma $= 54{,}598 + 7{,}389 + 1 = 62{,}987$

$$p_A \approx 0{,}867 \qquad p_B \approx 0{,}117 \qquad p_C \approx 0{,}016$$

Passo 3 (top-$p = 0{,}9$, com $T = 1$): $0{,}665 < 0{,}9$; $0{,}665 + 0{,}245 = 0{,}910 \geq 0{,}9$. O núcleo é $\{A, B\}$.

Temperatura menor aumentou a vantagem de A (de 0,665 para 0,867): o mesmo efeito que vimos com `␣Bel`.

Resolução

Passo 1 ($T = 1$): soma $= 7{,}389 + 2{,}718 + 1 = 11{,}107$

$$p_A \approx 0{,}665 \qquad p_B \approx 0{,}245 \qquad p_C \approx 0{,}090$$

Passo 2 ($T = 0{,}5$): escores viram $(4, 2, 0)$; soma $= 54{,}598 + 7{,}389 + 1 = 62{,}987$

$$p_A \approx 0{,}867 \qquad p_B \approx 0{,}117 \qquad p_C \approx 0{,}016$$

Passo 3 ($\textit{top-p} = 0{,}9$, com $T = 1$): $0{,}665 < 0{,}9$; $0{,}665 + 0{,}245 = 0{,}910 \geq 0{,}9$. O núcleo é $\{A, B\}$.

Temperatura menor aumentou a vantagem de A (de 0,665 para 0,867): o mesmo efeito que vimos com `␣Bel`.

Amostragem real: 20 sementes

Contexto: *"Um bom nome para um projeto de IA na UFPA Cametá é"*, com 10 tokens gerados e 20 sementes diferentes.

T	**distintas**	**exemplos**
0	1/20	*"CAMELIA" (Cametá*, sempre a mesma
0,7	19/20	*"NEXUS-UFPA". Neste · "Cametá Inteligente" ou "*
1,5	20/20	*"MindGeant" (A Mente É · : "Cem anos sem previsão:*

Leitura

Para tarefas factuais (extração, classificação), use temperatura baixa: a saída é reprodutível. Para ideias, uma temperatura moderada dá variedade; temperatura alta começa a produzir texto sem sentido.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Prompting e Limites

Zero-shot × *few-shot*

Tarefa: classificar comentários de estudantes como positivo, negativo ou misto.

Comentário	***Zero-shot*** (só a instrução)	***Few-shot***
A professora explica muito bem, mas as provas são longas demais.	"Eu classificaria esse comentário como 'misto'. Por um lado..."	misto
Adorei o laboratório de IA!	"O comentário é positivo."	positivo
O ar-condicionado da sala nunca funciona.	"Eu classificaria esse comentário como negativo. A afirmação..."	negativo

Leitura (Brown et al., 2020)

As duas versões acertaram as classes, mas só o *few-shot* devolveu **uma palavra**, pronta para ser lida por código. Os exemplos no *prompt* ensinam o **formato** e o rótulo esperado, sem treinar o modelo de novo.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Zero-shot × *few-shot*

Tarefa: classificar comentários de estudantes como positivo, negativo ou misto.

Comentário	***Zero-shot*** (só a instrução)	***Few-shot***
A professora explica muito bem, mas as provas são longas demais.	"Eu classificaria esse comentário como 'misto'. Por um lado..."	misto
Adorei o laboratório de IA!	"O comentário é positivo."	positivo
O ar-condicionado da sala nunca funciona.	"Eu classificaria esse comentário como negativo. A afirmação..."	negativo

Leitura (Brown et al., 2020)

As duas versões acertaram as classes, mas só o *few-shot* devolveu **uma palavra**, pronta para ser lida por código. Os exemplos no *prompt* ensinam o **formato** e o rótulo esperado, sem treinar o modelo de novo.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Cadeia de raciocínio (*chain-of-thought*)

"Uma disciplina tem 68 horas-aula. Cada aula dura 2 horas-aula e acontece 2 vezes por semana. Quantas semanas dura a disciplina?" (correto: **17**)

Resposta direta

"Responda apenas com o número." $\rightarrow$ **24**

"Pense passo a passo"

$68 \div 2 = 34$ aulas · $34 \div 2 = 17$ semanas · "Resposta: **17**"

Por que funciona (Wei et al., 2022)

Cada token gerado vira contexto para o próximo. Escrever os passos intermediários dá ao modelo "espaço" para calcular, em vez de saltar direto para um número provável. Testamos 3 problemas, repetindo cada um 6 vezes: o passo a passo acertou em todas as execuções; a resposta direta errou sempre em 2 deles e oscilou no terceiro.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Cadeia de raciocínio (*chain-of-thought*)

"Uma disciplina tem 68 horas-aula. Cada aula dura 2 horas-aula e acontece 2 vezes por semana. Quantas semanas dura a disciplina?" (correto: **17**)

Resposta direta

"Responda apenas com o número." $\rightarrow$ **24**

"Pense passo a passo"

$68 \div 2 = 34$ aulas · $34 \div 2 = 17$ semanas · "Resposta: **17**"

Por que funciona (Wei et al., 2022)

Cada token gerado vira contexto para o próximo. Escrever os passos intermediários dá ao modelo "espaço" para calcular, em vez de saltar direto para um número provável. Testamos 3 problemas, repetindo cada um 6 vezes: o passo a passo acertou em todas as execuções; a resposta direta errou sempre em 2 deles e oscilou no terceiro.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Papéis e saída estruturada

```
messages = [
  {"role": "system",
   "content": "Você extrai dados e
       responde apenas em JSON."},
  {"role": "user",
   "content": 'Extraia {"disciplina", "
       carga_horaria", "semestre"} de: "
       INTELIGÊNCIA ARTIFICIAL (68 horas-
       aula), ofertada no 7º semestre."'}]
```

Saída real (`format="json"`)

```
{"disciplina": "INTELIGÊNCIA ARTIFICIAL",
"carga_horaria": "68 horas-aula",
"semestre": "7º"}
```

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Boas práticas

A mensagem de **sistema** define o papel e as regras; a do **usuário** traz a tarefa. Pedir JSON torna a saída processável por código, mas valide os campos: o formato certo não garante o conteúdo certo.

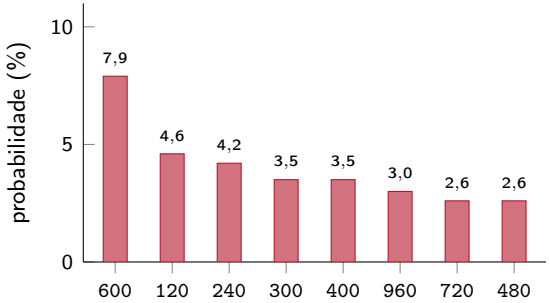
Alucinação: quando o provável não é o verdadeiro

Continuação de *"... da UFPA Campus Cametá é de"*: os candidatos para o número.

O favorito, 600, tem só **7,9%**. Uma distribuição tão espalhada costuma indicar que o modelo **não sabe**; mesmo assim, a saída gulosa escreve o número com a mesma fluência.

Valor real no PPC: **3.332 horas**.

execução real · llama3.2 (Ollama, CPU) · *seed* 42



Limitações e a ponte para o RAG

Limites de um LLM sozinho

- **Alucinação:** gera o provável, não o verificado.
- **Corte de conhecimento:** só sabe o que viu no treino. O *template* do `llama3.2` declara: *Cutting Knowledge Date: December 2023*.
- **Janela finita:** o que não cabe no contexto não existe para ele.
- **Sem fontes:** não diz de onde veio a resposta.

O que vimos na Aula 7

O RAG coloca evidências do acervo **dentro do contexto**. Com o trecho certo do PPC, a pergunta da carga horária foi respondida com **3.332 horas** e a página citada.

E agentes

Um agente deixa o LLM decidir **quando** buscar ou usar uma ferramenta (`Agentes.ipynb`).

Limitações e a ponte para o RAG

Limites de um LLM sozinho

- **Alucinação:** gera o provável, não o verificado.
- **Corte de conhecimento:** só sabe o que viu no treino. O *template* do `llama3.2` declara: *Cutting Knowledge Date: December 2023*.
- **Janela finita:** o que não cabe no contexto não existe para ele.
- **Sem fontes:** não diz de onde veio a resposta.

O que vimos na Aula 7

O RAG coloca evidências do acervo **dentro do contexto**. Com o trecho certo do PPC, a pergunta da carga horária foi respondida com **3.332 horas** e a página citada.

E agentes

Um agente deixa o LLM decidir **quando** buscar ou usar uma ferramenta (`Agentes.ipynb`).

Python: probabilidades do próximo token

```
import requests, math
r = requests.post("http://localhost:11434/api/generate", json={
    "model": "llama3.2", "prompt": "A capital do Pará é", "raw": True,
    "stream": False, "logprobs": True, "top_logprobs": 5,
    "options": {"num_predict": 1, "temperature": 0, "seed": 42,
                "num_gpu": 0}}).json()   # CPU: reproduz os números da aula
for c in r["logprobs"][0]["top_logprobs"]:
    print(repr(c["token"]), f'{math.exp(c["logprob"]):.3f}')
```

Saída: ’ Bel’ 0.752 · ’ a’ 0.208 · ’ o’ 0.013 · . . . O notebook `LLM.ipynb` traz todos os passos desta aula. Na GPU, os números mudam um pouco (ex.: ’ Bel’ sai com 0,775): as contas em ponto flutuante seguem outra ordem.

execução real · llama3.2 (Ollama, CPU) · *seed* 42

Referências Bibliográficas

- Shannon, C. E. *A Mathematical Theory of Communication*. Bell System Technical Journal, 1948.
- Bengio, Y. et al. *A Neural Probabilistic Language Model*. JMLR, 2003.
- Mikolov, T. et al. *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781, 2013.
- Sennrich, R.; Haddow, B.; Birch, A. *Neural Machine Translation of Rare Words with Subword Units*. ACL, 2016.
- Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017.
- Brown, T. et al. *Language Models are Few-Shot Learners*. NeurIPS, 2020.
- Kaplan, J. et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- Holtzman, A. et al. *The Curious Case of Neural Text Degeneration*. ICLR, 2020.
- Su, J. et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, 2021.
- Hoffmann, J. et al. *Training Compute-Optimal Large Language Models*. arXiv:2203.15556, 2022.
- Ouyang, L. et al. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022.
- Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 2022.
- Ainslie, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP, 2023.
- Grattafiori, A. et al. *The Llama 3 Herd of Models*. arXiv:2407.21783, 2024.
- Jurafsky, D.; Martin, J. H. *Speech and Language Processing*. 3. ed. (rascunho on-line), cap. 7–10.

Modelos de Linguagem de Grande Escala (LLMs)

Aula 8: como um LLM funciona por dentro

Prof. Elton Sarmanho[1]
eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA Campus Cametá

18 de outubro de 2026